

20 questions Python pandas

Manipuler un DataFrame /20



Comme un analyste face à un vrai fichier de ventes. Durée conseillée : 20 min · 3 paliers · corrigé détaillé.

Objectif : mesurer tes réflexes pandas réels, pas ta syntaxe récitée. Ce test vérifie si tu sais quand un `merge` duplique tes lignes, quel type pandas a vraiment choisi pour une colonne, et pourquoi un total peut devenir faux sans qu'aucune erreur ne s'affiche.

Sur les fichiers de ventes que j'ouvre en clientèle, la moitié des bugs viennent d'une date mal lue ou d'un `merge` qui double des lignes en silence, jamais d'une syntaxe pandas mal écrite.

Repères : 12/20 = junior, 17/20 = profil qu'on garde.

Candidat _____ Date _____ Note _____ /20

Palier 1 : lire et trier un fichier de ventes.

Sept questions pour vérifier la base : chargement d'un CSV, sélection par label ou position, tri, valeurs manquantes.

Question 1. Tu reçois `export_ventes.csv` depuis l'ERP et tu charges avec `pd.read_csv('export_ventes.csv')`. Que fait pandas par défaut sur le séparateur et les types de colonnes ?

- a) Il détecte automatiquement le séparateur, virgule ou point-virgule.
- b) Il force toutes les colonnes en texte (object) pour éviter les erreurs.
- c) Il suppose une virgule comme séparateur et infère le type de chaque colonne, avec un repli en object si une colonne mélange les types.
- d) Il refuse de charger le fichier si une colonne contient à la fois des nombres et du texte.

Question 2. Sur `ventes` trié par ligne, tu veux les lignes d'index 2 à 5 inclus avec `.iloc[2:5]`. Que se passe-t-il si tu écris `.iloc[2:5]` à la place ?

- a) `.iloc[2:5]` inclut la ligne d'index 5, alors que `.iloc[2:5]` s'arrête avant la position 5 (exclusive).
- b) `.loc` et `.iloc` se comportent de façon identique sur un slice numérique.
- c) `.iloc[2:5]` inclut la position 5, contrairement à `.loc[2:5]`.
- d) `.loc` lève une erreur si la borne finale n'existe pas dans l'index.

Question 3. Un collègue insiste : `isna()` détecte les NaN, `isnull()` détecte les None, ce ne sont pas les mêmes valeurs manquantes. Qu'en penses-tu ?

- a) Il a raison, il faut toujours choisir `isna()` pour les colonnes numériques.
- b) Il a raison, `isnull()` est réservé aux colonnes de type objet.
- c) Il a tort : seul `isna()` existe en pandas, `isnull()` a été supprimé.
- d) Il a tort : `isna()` et `isnull()` sont des alias strictement identiques, aucune différence de comportement.

Question 4. La colonne `montant` contient quelques valeurs manquantes (ventes non facturées). Tu tries avec `ventes.sort_values('montant')`. Où atterrissent les lignes sans montant ?

- a) En tête de tableau, avant les montants les plus faibles.
- b) En fin de tableau, après les montants les plus élevés, quel que soit le sens du tri.
- c) Elles sont supprimées automatiquement par `sort_values`.
- d) Réparties aléatoirement dans le tableau trié.

Question 5. Tu exécutes `ventes['region'].value_counts()` sur une colonne où plusieurs dizaines de ventes n'ont pas de région renseignée. Que contient le résultat ?

- a) Les régions triées par ordre alphabétique, NaN inclus comme une catégorie.
- b) Les régions triées par effectif décroissant, sans la ligne des valeurs manquantes.
- c) Une seule ligne avec le total de toutes les régions confondues.
- d) Une erreur, `value_counts` refuse les colonnes contenant des NaN.

Question 6. La table `clients` contient deux lignes identiques pour « Martin SARL », saisies à deux dates différentes. Tu appelles `clients.drop_duplicates(subset='nom')`. Laquelle des deux lignes est conservée par défaut ?

- a) La ligne avec la date la plus récente.
- b) Les deux lignes sont supprimées, aucune n'est conservée.
- c) Une ligne choisie aléatoirement à chaque exécution.
- d) La première occurrence rencontrée dans l'ordre du fichier ; la ou les suivantes sont supprimées.

Question 7. La colonne `commentaire_client` est vide sur 70% des lignes de ventes. Un collègue propose `ventes.dropna()` pour « nettoyer » le fichier avant l'analyse du chiffre d'affaires. Quel est le risque ?

- a) `dropna()` supprime toute ligne où au moins une colonne est NaN : il élimine 70% des ventes valides à cause d'un commentaire vide, faussant le chiffre d'affaires total.
- b) Aucun risque, `dropna()` ne cible que les colonnes entièrement vides.
- c) `dropna()` remplace automatiquement les valeurs manquantes par 0 avant de calculer le total.
- d) `dropna()` ne s'applique qu'aux colonnes numériques, la colonne texte n'est pas concernée.

Palier 2 : le quotidien de l'analyste.

Sept questions sur ce qui arrive tous les jours sur un vrai fichier : fusionner, grouper, croiser, parser une date. C'est là qu'un total se met à mentir sans qu'aucune erreur ne s'affiche.

Question 8. Tu reçois deux exports mensuels de ventes avec les mêmes colonnes, et tu veux les mettre bout à bout dans un seul DataFrame. Que dois-tu utiliser ?

- a) `merge()`, pour aligner les deux tables sur une clé commune.
- b) `merge(how='outer')`, qui empile toutes les lignes des deux tables.
- c) `concat([ventes_janvier, ventes_fevrier])`, qui empile verticalement les deux tables en alignant les colonnes par nom.
- d) `join()`, réservé aux jointures sur l'index uniquement.

Question 9. Certaines lignes de ventes n'ont pas de région renseignée (NaN). Tu exécutes `ventes.groupby('region')['montant'].sum()`. Le résultat contient-il une ligne pour les ventes sans région ?

- a) Oui, sous la clé 'NaN' littérale.
- b) Oui, sous la clé 'inconnu' générée automatiquement.
- c) Non, une erreur est levée car `groupby` refuse les clés manquantes.
- d) Non : `groupby` exclut par défaut les groupes dont la clé est NaN (`dropna=True`), ces ventes disparaissent silencieusement du total.

Question 10. PIÈGE CLASSIQUE. Tu fais `ventes.merge(clients, on='client_id', how='left')` pour enrichir tes ventes. La table `clients` contient par erreur deux lignes pour le `client_id` 482 (doublon de saisie). Que se passe-t-il sur les ventes de ce client ?

- a) pandas ignore le doublon et ne garde que la première correspondance.
- b) Chaque vente du client 482 est dupliquée en deux lignes, une par correspondance trouvée côté droit : le nombre total de lignes augmente.
- c) La fusion échoue avec une erreur de clé dupliquée.
- d) pandas fusionne les deux lignes clients en une seule avant la jointure.

Question 11. Tu écris `ventes.groupby(['region', 'annee']).agg({'montant': 'sum'})`. Que produit ce code ?

- a) Un DataFrame avec le total des montants pour chaque combinaison région/année, indexé par ces deux colonnes (multi-index) si `as_index` reste à sa valeur par défaut.
- b) Une erreur, `agg()` n'accepte pas plusieurs colonnes de regroupement.
- c) Une seule ligne avec le total général, région et année étant ignorées.
- d) Un DataFrame trié automatiquement par montant décroissant.

Question 12. Tu construis `ventes.pivot_table(index='region', columns='mois', values='montant')` sans préciser `aggfunc`, en pensant obtenir la somme des ventes par région et par mois. Le résultat te semble anormalement bas. Pourquoi ?

- a) `pivot_table` calcule toujours le nombre de lignes (count), jamais un montant.
- b) `pivot_table` additionne automatiquement, le résultat est correct.
- c) La colonne montant est convertie en texte par défaut.
- d) L'aggfunc par défaut de `pivot_table` est 'mean' : le tableau affiche une moyenne par case, pas une somme, sans avertissement.

Question 13. Le fichier `export_ventes.csv` contient une colonne date au format français 03/04/2024 (le 3 avril). Tu écris `pd.to_datetime(ventes['date'])` sans autre argument. Que se passe-t-il ?

- a) pandas détecte automatiquement le format français DD/MM/YYYY.
- b) pandas interprète 03/04/2024 en mode mois d'abord (`dayfirst=False` par défaut) : la date devient le 4 mars, pas le 3 avril.
- c) La conversion échoue avec une erreur bloquante sur tout format ambigu.
- d) pandas convertit uniquement les dates dont le jour dépasse 12, laissant les autres inchangées.

Question 14. La colonne `commentaire_client` contient des NaN sur les lignes sans commentaire. Tu filtres avec `ventes[ventes['commentaire_client'].str.contains('remboursement')]`. Que se passe-t-il ?

- a) Les lignes sans commentaire sont automatiquement traitées comme ne contenant pas 'remboursement', donc exclues proprement.
- b) Le filtre s'exécute normalement et retourne un DataFrame vide.
- c) `str.contains` renvoie NaN pour les lignes sans commentaire, et le filtrage lève une erreur car un masque booléen ne peut pas contenir de NaN, sauf à passer `na=False`.
- d) Les NaN sont convertis en chaîne vide automatiquement avant la comparaison.

Palier 3 : les pièges qui trient les juniors.

Six questions sur ce qui sépare un candidat qui connaît pandas d'un candidat qui l'a vraiment pratiqué : copie ou vue, performance, typage mémoire.

Question 15. Un collègue calcule une marge par ligne avec `ventes.apply(lambda row: row['prix'] - row['cout'], axis=1)` sur un fichier de 500 000 lignes, et trouve ça très lent. Que lui conseilles-tu ?

- a) Remplacer par `ventes['prix'] - ventes['cout']`, une opération vectorisée bien plus rapide que `apply(axis=1)` qui appelle la fonction ligne par ligne.
- b) Passer à `axis=0`, qui traite les lignes plus vite que `axis=1`.
- c) Le problème vient de pandas, `apply` est toujours plus rapide qu'une soustraction directe de colonnes.
- d) Ajouter `inplace=True` à l'appel `apply` pour accélérer le calcul.

Question 16. La colonne `prix` contient une valeur saisie par erreur en texte ('non renseigné') au milieu de nombres. Tu veux la convertir en numérique en transformant cette valeur invalide en NaN, sans faire planter le script. Quelle instruction fait exactement ça ?

- a) `ventes['prix'].astype(float)`, qui convertit automatiquement les textes invalides en NaN.
- b) `ventes['prix'].astype(float, errors='coerce')`, option native de `astype` pour ce cas.
- c) `ventes['prix'].fillna(0).astype(float)`, qui règle le problème en amont.
- d) `pd.to_numeric(ventes['prix'], errors='coerce')`, qui convertit ce qui est convertible et remplace les valeurs invalides par NaN.

Question 17. PIÈGE ÉLIMINATOIRE. Tu écris `grosses_ventes = ventes[ventes['montant'] > 1000]` puis `grosses_ventes['statut'] = 'à valider'`. Python affiche un `SettingWithCopyWarning`. Que signifie cet avertissement ?

- a) L'opération a échoué et aucune colonne n'a été modifiée.
- b) `grosses_ventes` peut être une vue sur `ventes` plutôt qu'une copie garantie : la modification risque de ne pas se répercuter comme attendu, ou d'affecter silencieusement l'original.
- c) Il s'agit d'un simple avertissement de style à ignorer, sans conséquence sur les données.
- d) La colonne `statut` n'existait pas encore dans `ventes`, d'où l'avertissement.

Question 18. La colonne `statut` de ventes ne prend que 4 valeurs possibles ('payée', 'en attente', 'annulée', 'remboursée') répétées sur 2 millions de lignes. Quelle transformation réduit la mémoire utilisée sans changer les valeurs affichées ?

- a) `ventes['statut'].astype(str)`, qui force un typage texte optimisé.
- b) `ventes.drop(columns='statut')` puis recréation en dur.
- c) `ventes['statut'].astype('category')`, qui stocke chaque valeur unique une seule fois et remplace les répétitions par des références internes.
- d) `ventes['statut'].astype(int)`, qui convertit le texte en identifiants numériques visibles.

Question 19. Après un `groupby('region').sum()`, tu veux remettre `region` comme colonne normale (et non comme index) pour l'exporter proprement en CSV. Tu appelles `resultat.reset_index()`. Que se passe-t-il par défaut ?

- a) L'ancien index (`region`) redevient une colonne classique, insérée en première position du DataFrame.
- b) L'ancien index est définitivement perdu, seules les colonnes de valeurs restent.
- c) `reset_index()` ne fonctionne que si l'argument `drop=True` est précisé.
- d) Un nouvel index numérique est créé, mais aucune colonne `region` n'apparaît dans le résultat.

Question 20. Tu veux créer une sous-table indépendante à partir de `ventes`, que tu pourras modifier librement sans jamais toucher au fichier original ni déclencher de `SettingWithCopyWarning`. Quelle instruction le garantit ?

- a) `sous_table = ventes[ventes['region']=='Nord']`, utilisée telle quelle pour toute modification.
- b) `sous_table = ventes.loc[ventes['region']=='Nord']`, sans rien ajouter d'autre.
- c) `sous_table = ventes[ventes['region']=='Nord'].copy()`, qui crée un objet totalement indépendant, explicitement détaché de `ventes`.
- d) `sous_table = ventes`; puis filtrage direct sur `sous_table`.

Solutions et explications.

Un point par bonne réponse. Compte ton score, puis relis surtout celles que tu ne peux pas expliquer à voix haute.

- 1. c)** `pd.read_csv` suppose une virgule comme séparateur (pas de détection automatique) et infère le type de chaque colonne, avec `repl` en object dès qu'une colonne mélange nombres et texte. Piège classique : croire que le séparateur est deviné, alors qu'un export Excel en point-virgule charge tout dans une seule colonne sans erreur visible.
- 2. a)** `.loc` travaille sur les labels et inclut la borne finale; `.iloc` travaille sur les positions et l'exclut, comme un slice Python classique. Piège classique : copier un slice `.iloc` et l'appliquer à `.loc` en pensant obtenir la même plage.
- 3. d)** `isnull()` est un alias historique conservé pour compatibilité : les deux méthodes appellent le même code et renvoient exactement le même résultat. Piège classique : chercher une nuance technique entre les deux qui n'existe pas.
- 4. b)** `sort_values` trie par défaut en ordre croissant et place les valeurs manquantes en dernier (`na_position='last'`), même en `ascending=False`. Piège classique : croire qu'un tri décroissant remonte les NaN en haut.
- 5. b)** `value_counts` trie par défaut du plus fréquent au moins fréquent et exclut les NaN, sauf `dropna=False`. Piège classique : ne voir nulle part les ventes sans région et conclure à tort que toutes les régions sont renseignées.
- 6. d)** `drop_duplicates()` utilise par défaut `keep='first'` : la première ligne rencontrée dans l'ordre du fichier est gardée. Piège classique : supposer que pandas garde « la plus récente », alors qu'il ne regarde que l'ordre d'apparition.
- 7. a)** `dropna()` supprime par défaut toute ligne contenant au moins un NaN sur l'ensemble des colonnes. Piège classique : appliquer un `dropna()` global « pour nettoyer » sans `subset`, et supprimer des ventes réelles à cause d'une colonne annexe vide.
- 8. c)** `concat(axis=0)` empile les lignes verticalement en alignant les colonnes par nom, l'opération recherchée pour deux exports de même structure. Piège classique : utiliser `merge`, qui cherche une clé de jointure et peut dupliquer ou perdre des lignes sans qu'aucune jointure ne soit nécessaire.
- 9. d)** Depuis pandas 1.1, `groupby` a `dropna=True` par défaut : toute ligne dont la colonne de regroupement est NaN est exclue, sans avertissement. Piège classique : sommer le résultat et le comparer au total général sans remarquer que des ventes manquent à l'appel.
- 10. b)** `how='left'` garde toutes les lignes de gauche, mais si la clé de droite contient des doublons, chaque ligne de gauche est répétée une fois par correspondance : le nombre de lignes explose sans erreur. Piège classique : ne pas vérifier `.duplicated()` côté droit avant de fusionner.
- 11. a)** `groupby` sur plusieurs colonnes crée un groupe par combinaison de valeurs ; avec `as_index=True` par défaut, `region` et `annee` deviennent un index à deux niveaux. Piège classique : oublier ce multi-index et filtrer ensuite comme sur une colonne normale.
- 12. d)** Sans `aggfunc` explicite, `pivot_table` calcule la moyenne (`mean`), pas la somme. Piège classique : lire un tableau croisé et supposer qu'il montre des totaux, alors qu'un mois avec peu de ventes mais un montant moyen élevé inverse l'analyse.
- 13. b)** `to_datetime` a pour défaut `dayfirst=False` (convention mois d'abord) ; sur une date française ambiguë, la conversion se fait sans erreur mais avec jour et mois inversés. Piège classique : ne repérer l'erreur qu'en aval, sur des pics inexplicables en début d'année.
- 14. c)** `str.contains` propage le NaN par défaut : une case vide donne NaN, pas `False`, et un masque avec des NaN dedans provoque une erreur au filtrage. Piège classique : oublier `na=False` sur une colonne texte partiellement vide et voir le script planter en production.
- 15. a)** `axis=1` dans `apply` appelle la fonction une fois par ligne, nettement plus lent qu'une opération vectorisée sur les colonnes. Piège classique : réflexe « apply pour tout » hérité d'Excel, alors qu'une soustraction de colonnes fait le même calcul en une fraction du temps.
- 16. d)** `pd.to_numeric` avec `errors='coerce'` convertit ce qui est convertible et remplace le reste par NaN sans planter. Piège classique : croire que `astype(float)` accepte un `errors='coerce'` comme `to_numeric` ; `astype` ne connaît que `errors='raise'` (défaut, qui plante) ou `errors='ignore'`, qui ne convertit rien.
- 17. b)** Un filtrage booléen suivi d'une assignation directe crée une ambiguïté : pandas ne garantit pas si le résultat est une vue ou une copie. Piège éliminatoire : ignorer ce warning comme cosmétique, alors que l'effet de la modification devient imprévisible ; la correction propre est un `.copy()` explicite après le filtrage.
- 18. c)** Le type `category` stocke une seule fois chaque valeur distincte et remplace les répétitions par des références internes, ce qui réduit fortement la mémoire sur une colonne à faible cardinalité. Piège classique : laisser ce type de colonne en object sur un gros fichier et voir le script ralentir sans comprendre pourquoi.
- 19. a)** `reset_index()` a pour défaut `drop=False` : l'ancien index redevient une colonne normale, insérée en première position. Piège classique : appeler `reset_index(drop=True)` par réflexe et perdre silencieusement la colonne `region` au moment de l'export.
- 20. c)** `.copy()` force la création d'un objet mémoire indépendant, garanti détaché de l'original. Piège classique : s'arrêter au filtrage seul, qui peut renvoyer une vue partagée avec l'original, exactement la source de l'avertissement vu en question 17.

Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

